PriceOS Pricing Engine Customizations

PriceOS Pricing Engine Customizations

Owner/Reviewer: @Harshit Choudhary

Competitive Analysis vs PriceLabs + Prioritized Build Roadmap

---

How PriceLabs Structures Their Pricing Engine

PriceLabs has ~25+ customizations organized into layers. Their pricing calculation follows this order:

Base Price (user-set average annual rate)
→ Seasonal adjustments applied
→ Day-of-week factors applied
→ Lead-time factors (far-out premiums, last-minute discounts)
→ Demand/event multipliers
→ Occupancy-based adjustments
→ Orphan day / gap pricing
→ Adjacent booking factors
→ LOS discounts (weekly/monthly)
→ Guardrails (min/max price floors and ceilings)

The final price is the result of all these layers stacking. PriceLabs lets users toggle each layer independently and override at listing, group, sub-group, or account level.

The Full PriceLabs Customization Map

I've categorized every PriceLabs customization and scored each on **Revenue Impact** (how much it actually moves the needle for a Dubai STR portfolio) and **User Demand** (how often property managers ask for it).

TIER 1: Must Build (Highest Revenue Impact)

These are the customizations that PriceLabs users rely on daily. Without these, PriceOS is not a serious pricing tool.

1. Base Price + Min/Max Price Guardrails

- What it does: Sets the annual average rate, hard floor, and hard ceiling per unit
- PriceLabs approach: User sets manually; PriceLabs offers "Base Price Help" tool that analyzes nearby comps and suggests a starting point; "Nudges" alert users when base price drifts >5% from recommendation
- Revenue impact: 10/10. Everything else is a modifier on top of this. Get this wrong and nothing else matters
- PriceOS angle: Your onboarding spec already handles this in Stage 2 (guardrails). But add the intelligence layer: auto-calculate suggested base price from comp set ADR data. This was also called out in your product feedback. PriceLabs makes users do this manually or rely on a basic tool. PriceOS should calculate it automatically from scraped OTA data as part of onboarding

2. Seasonal Profiles (Custom Min/Base/Max per Season)

- What it does: Lets users define named seasons (e.g., "Peak Winter Dec-Mar", "Summer Low Jun-Aug", "Shoulder Sep-Nov") with different base, min, and max prices per season
- PriceLabs approach: Users create seasonal profiles covering Jan 1 to Dec 31. Can override min, base, and max per season. If left blank, defaults to listing-level prices
- Revenue impact: 9/10. Dubai has extreme seasonality: Dec-Mar is peak (up to 2x rates), Jun-Aug is trough (rates drop 40-50%). Operators who don't adjust seasonally leave massive money on the table in winter and sit empty in summer
- PriceOS angle: This aligns with the "+30% high season, -50% summer" feedback from your product doc. Build named seasons with percentage or

fixed overrides. Auto-suggest Dubai seasonal bands based on historical demand data. This should be a first-class feature, not buried in settings

3. Last-Minute Pricing (Gradual Discount Curve)

- What it does: Automatically reduces price as check-in date approaches for unbooked nights
- PriceLabs approach: Default is gradual 30% discount over next 15 days (50% on day 1, tapering to 5% on day 15). Users can customize to flat %, gradual %, fixed price, or turn off entirely. Discount never goes below minimum price
- Revenue impact: 9/10. In Dubai STR, last-minute bookings are a huge segment (business travelers, weekend getaways, spontaneous tourists). The difference between a vacant night and a discounted night at AED 400 instead of AED 600 is pure profit
- PriceOS angle: Build the gradual discount curve with configurable parameters: discount depth, ramp period, and floor protection. Your Pricing Agent should propose last-minute adjustments daily. PriceLabs is static here (set and forget). PriceOS can be dynamic: adjust the discount curve based on real-time booking velocity and comp availability

4. Occupancy-Based Adjustments (Listing + Portfolio Level)

- What it does: Automatically raises or lowers prices based on how booked the listing (or the entire portfolio/group) is within specific booking windows
- PriceLabs approach: Two versions. Listing-level: if your unit is 80% booked for next 30 days, raise prices; if 30% booked, discount. Portfolio-level: looks at all units in a group and adjusts collectively. Uses configurable "profiles" with different adjustments per booking window (0-7 days, 8-14 days, 15-30 days, etc.)
- Revenue impact: 9/10. This is the single most powerful revenue lever for multi-unit operators. If your Marina 1-beds are 90% booked next month but your JVC properties are at 40%, portfolio-level occupancy adjustment automatically shifts pricing strategy per zone
- PriceOS angle: You already have the Portfolio Analyzer agent. Wire it to occupancy-based pricing rules. The key differentiation: PriceLabs requires manual profile setup. PriceOS should auto-calibrate based on observed booking velocity. When the system sees high velocity, it raises prices

automatically; when velocity stalls, it discounts. This is the "AI" in your revenue manager

5. Orphan Day / Gap Pricing

- What it does: Adjusts pricing and minimum stay for gaps between existing bookings that are too short to sell normally
- PriceLabs approach: Default 20% discount on 1-2 night gaps. Users can set up to 5 gap range rules (e.g., 1-night gaps at -10%, 2-night gaps at -15%, 3-4 night gaps at -5%). Can set different rules for weekdays vs weekends. Options: fixed price, percentage discount, or percentage premium
- Revenue impact: 8/10. Orphan gaps are the #1 silent revenue killer in STR. A property manager with 50 units might have 15-20 orphan gaps at any given time. Each one is AED 400-800 in lost revenue per night
- PriceOS angle: Already in your onboarding spec (Gap & LOS Optimizer agent). Build multi-rule gap pricing: configurable by gap length, days until check-in, and weekday/weekend. The AI layer should proactively detect orphan gaps forming and suggest pricing + minimum stay changes before they become unfillable

6. Day-of-Week Pricing Adjustments

- What it does: Applies percentage adjustments per day of week on top of base recommendations
- PriceLabs approach: Percentage adjustments from -75% to +500% per day. Their algorithm already has day-of-week factors built in, so this is an additional layer. Key Dubai consideration: weekends are Thursday-Friday in some contexts (though Dubai has shifted to Sat-Sun officially, behavior still varies)
- Revenue impact: 7/10. Thursday and Friday nights in Dubai are consistently higher demand than Monday-Wednesday. PMs who don't adjust leave 10-15% ADR uplift on weekends
- PriceOS angle: Build configurable weekend day selection (critical for Middle East). Auto-detect day-of-week patterns from booking history rather than requiring manual percentage input. Your Market Intel Agent should surface these patterns

TIER 2: Should Build (Strong Revenue Impact, Differentiating)

7. Far-Out Minimum Price Protection

- What it does: Sets a higher minimum price floor for dates far in the future to prevent selling peak dates too cheaply
- PriceLabs approach: "For any date more than X days out, don't let the price drop below Y." Options: fixed price, % of base, or % of minimum. Protects high-season inventory from being sold at low-season rates months in advance
- Revenue impact: 8/10. Massive for Dubai. A New Year's Eve booking made in August should not be sold at October rates. Property managers consistently complain about this
- PriceOS angle: This is a guardrail enhancement. Add a "booking window floor" that increases minimum prices as the date gets further out. Your Revenue Optimizer agent should enforce this: "This date is 120 days out and is during Dubai Shopping Festival. Do not price below AED 800"

8. Minimum Weekend Pricing

- What it does: Separate minimum price for weekend nights (above the standard minimum)
- PriceLabs approach: Fixed price, % on base, or % on min. Works with custom weekend day definition. Stacks with far-out minimum pricing
- Revenue impact: 7/10. Prevents weekend nights from being discounted below a profitable threshold. Especially important in Dubai where weekend demand is structurally higher
- PriceOS angle: Simple addition to the guardrails engine. Weekend min should be a separate field per unit/group, not a global setting

9. Date-Specific Overrides (DSO)

- What it does: Manual override of price and/or minimum stay for specific dates (events, holidays, personal blocks)
- PriceLabs approach: Override options for min, max, base, or recommended price by fixed amount or percentage. Also supports min stay overrides per date. Can be applied at listing, group, or account level
- Revenue impact: 8/10. Every experienced PM in Dubai knows: Formula 1 week, New Year's, Dubai World Cup, Ramadan, National Day all need hand-tuned pricing. The AI can suggest, but the user must be able to override

- PriceOS angle: Already covered in your feedback doc ("users must always be able to override any automated pricing decision"). Build a calendar-level override UI. This is table stakes

10. Adjacent Booking Factor

- What it does: Adjusts price for nights immediately before or after an existing booking
- PriceLabs approach: Configurable: discount (to encourage fill) or premium (to discourage same-day turnovers for properties with high cleaning costs). 1-30 day range. By default doesn't apply to weekends. Stacks with orphan and last-minute pricing using priority rules
- Revenue impact: 6/10. More niche, but valuable for operators who want to control turnover patterns. Dubai serviced apartments with expensive deep cleans benefit from a turnaround premium
- PriceOS angle: Build as part of Gap & LOS Optimizer. Add a "turnaround cost" parameter per property so the system automatically prices adjacent days to cover operational costs

11. Length of Stay Discounts (Weekly/Monthly)

- What it does: Applies percentage discounts for longer bookings (7+ nights, 28+ nights)
- PriceLabs approach: Weekly and monthly discount percentages pushed to OTA platforms. Applied after algorithm runs, meaning they can push final rate below minimum price
- Revenue impact: 7/10. Dubai has significant mid-term rental demand (1-3 month corporate relocations, project-based stays). Offering a structured LOS discount captures this segment
- PriceOS angle: Build tiered LOS pricing: 7-night discount, 14-night discount, 28-night discount. Make it configurable per unit/group. The CRO should suggest LOS tiers based on observed booking patterns

TIER 3: Nice to Have (Incremental, Build Later)

12. Demand Factor Sensitivity (Hotel Weight)

- What it does: Controls how much influence nearby hotel pricing has on STR rates

- PriceLabs approach: Slider from 0% (fully STR) to 100% (fully hotel). Default 10%. Can select specific hotels for comparison
- Revenue impact: 5/10. Useful for apart-hotels and serviced apartments in Dubai that compete more with hotels than Airbnbs
- Build later: After core engine is solid. Low urgency for beta

13. Customization Hierarchy (Account > Group > Sub-Group > Listing)

- What it does: Allows bulk settings at portfolio level with increasingly specific overrides
- PriceLabs approach: Four-tier hierarchy. Most specific setting wins. Percentage-based group settings are preferred for mixed portfolios
- Revenue impact: 6/10 for portfolio operators, 2/10 for small PMs
- Build later: Start with listing-level and group-level. Add sub-groups when you have operators with 100+ units

14. Check-in/Check-out Day Restrictions

- What it does: Restricts which days guests can check in or check out
- PriceLabs approach: Configurable per listing or season. Common use: Friday check-in only, Sunday check-out only for resort properties
- Revenue impact: 4/10 in Dubai context (less relevant for urban STR)
- Build later: Not a priority for Dubai Marina/Downtown portfolio types

15. Pricing Offsets (Channel-Specific)

- What it does: Applies a markup or discount to specific booking channels (e.g., +8% on VRBO because of higher OTA fees)
- PriceLabs approach: Percentage or fixed offset per channel for mapped listings
- Revenue impact: 5/10. Relevant for multi-channel distribution
- Build later: Useful but not a differentiator. Most PMS tools handle this

16. Intra-Day Portfolio Occupancy Adjustments

- What it does: Applies different pricing adjustments at different times of day for today and tomorrow to capture same-day booking demand

- PriceLabs approach: Multiple adjustment windows within a single day. E.g., higher discount at 6PM for tonight's vacancy than at 9AM
- Revenue impact: 4/10 currently but growing as same-day booking increases
- Build later: Advanced feature for V2

The PriceOS Competitive Edge (What PriceLabs Can't Do)

PriceLabs is fundamentally a **rules-based engine with manual configuration**. Users set parameters, the algorithm applies them. Here's where PriceOS should differentiate:

1. Agent-driven intelligence vs static rules
PriceLabs requires the user to know what settings to use. PriceOS agents should observe, learn, and recommend. Example: "I notice your Marina 1-beds consistently get booked 3 weeks before check-in. Your far-out minimum is too low for dates 30+ days out. Recommend raising it from AED 400 to AED 550."

2. Simulation before execution
PriceLabs pushes prices immediately. PriceOS shows what it would do first (your simulation stage). This is a massive trust builder that PriceLabs doesn't offer.

3. Dubai-native market intelligence
PriceLabs applies generic seasonality. PriceOS should have hardcoded Dubai intelligence: Ramadan demand patterns, DTCM event calendar integration, zone-level micro-seasonality (Marina vs JVC vs Downtown behave differently), and awareness that weekends function differently in the UAE.

4. Natural language pricing management
PriceLabs requires clicking through settings panels. PriceOS lets users say: "Raise all Marina properties by 15% for New Year's week" through the CRO. This is the conversational layer that makes PriceOS feel like having a revenue manager, not using a software tool.

5. Automatic base price calibration
PriceLabs suggests a base price through a basic tool. PriceOS should auto-calculate ADR from scraped OTA data and dynamically recalibrate every month, surfacing nudges when the base drifts.

Recommended Build Order for PriceOS

Phase 1: Foundation (Sprint 1-2)

Already in your sprint plan. Gets the core engine working.

1. Base Price + Min/Max Guardrails (with auto-ADR calculation)
2. Seasonal Profiles (named seasons with % or fixed overrides)
3. Day-of-Week Adjustments (with UAE weekend configuration)
4. Date-Specific Overrides (manual user control on any date)
5. Bar-Level Rate Grid (from your product feedback)

Phase 2: Intelligence Layer (Sprint 3-4)

The features that make PriceOS "AI-powered" rather than just "dynamic pricing."

1. Last-Minute Discount Curve (gradual, configurable, with velocity awareness)
2. Occupancy-Based Adjustments (listing + portfolio level)
3. Orphan Day / Gap Pricing (multi-rule, weekday/weekend split)
4. Far-Out Minimum Price Protection
5. Minimum Weekend Pricing

Phase 3: Advanced (Sprint 5-6)

Portfolio-scale features and competitive differentiation.

1. Adjacent Booking Factor
2. Length of Stay Discounts (weekly/monthly tiers)
3. Competitor Rate Comparison with auto-positioning
4. Portfolio-Level Occupancy Adjustments with booking window segmentation
5. Customization Hierarchy (account > group > listing)

Phase 4: V2 Differentiation (Post-Launch)

1. Intra-day pricing adjustments
2. Channel-specific offsets

3. Demand factor sensitivity / hotel weighting
4. Check-in/check-out day restrictions
5. Automatic base price recalibration with monthly nudges

Summary: The 80/20

If we build just the top 6 customizations from Tier 1, you cover ~80% of the revenue impact that PriceLabs delivers with 25+ customizations. our agents provide the remaining value through intelligence, not configuration complexity.

The PriceLabs user spends 30-60 minutes setting up customizations per listing. The PriceOS user should spend 5 minutes in onboarding and let the agents handle the rest, with the ability to override anything at any time.

That's our moat: **same revenue impact, 90% less configuration effort, with an AI layer that learns and adapts.**
